

DOCUMENTO DE ARQUITECTURA DE SOFTWARE SAD

SISTEMA DE GESTIÓN DE TURNOS Y TRIAJE MÉDICO AUTOMATIZADO

CARAL DANIELA MARTÍNEZ PANQUEVA

CC 1127045653

INGENIERÍA DE SISTEMAS

FACULTAD DE INGENIERÍAS Y ARQUITECTURA

UNIVERSIDAD DE PAMPLONA

JUNIO, 2026

1. INTRODUCCIÓN

1.1 Propósito

El presente documento describe la arquitectura de software del sistema MedTriage, definiendo su estructura interna, los componentes principales y las decisiones tecnológicas que guían su diseño e implementación.

1.2 Alcance

Este documento aplica al sistema de gestión de turnos y triaje médico automatizado desarrollado para el servicio de urgencias de Villa del Rosario, Norte de Santander. Cubre la arquitectura del frontend, el backend y la base de datos.

1.3 Definiciones Clave

- ✓ **MVC:** Patrón arquitectónico Modelo-Vista-Controlador.
- ✓ **Frontend:** Capa de presentación accesible desde el navegador web.
- ✓ **Backend:** Capa de lógica de negocio y procesamiento del servidor.
- ✓ **API REST:** Interfaz de comunicación entre frontend y backend.
- ✓ **PostgreSQL:** Motor de base de datos relacional utilizado por el sistema.

2. REPRESENTACIÓN ARQUITECTÓNICA

2.1 Estilo Arquitectónico

MedTriage adopta el patrón arquitectónico Modelo-Vista-Controlador (MVC), implementado sobre una arquitectura cliente-servidor. El frontend web actúa como capa de Vista, el backend en Spring Boot gestiona los Controladores y la lógica de negocio, y PostgreSQL constituye la capa de Modelo.

2.2 Diagrama de Contexto (C4 – Nivel 1)

El sistema MedTriage interactúa con cuatro tipos de usuarios: Admisión, Enfermería, Médico y Administrador, quienes acceden a través de un navegador web estándar. Adicionalmente, la pantalla pública de turnos es visible para pacientes y acompañantes sin autenticación.

Diagrama de Contexto (C4 – Nivel 1)

El sistema MedTriage interactúa con cuatro tipos de usuarios: Admisión, Enfermería, Médico y Administrador, quienes acceden a través de un navegador web estándar. Adicionalmente, la pantalla pública de turnos es visible para pacientes y acompañantes sin autenticación.

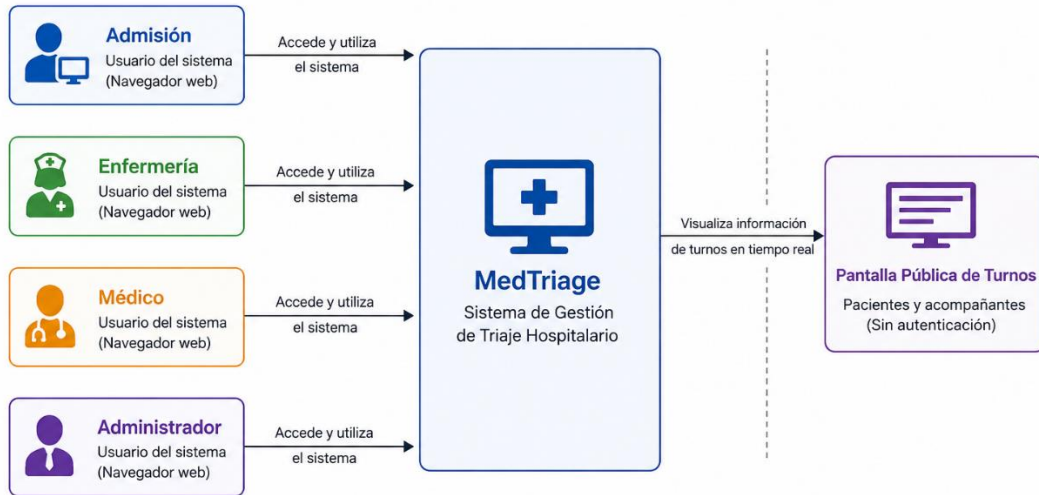


Figura 1 Diagrama de Contexto

2.3 Diagrama de Contenedores (C4 – Nivel 2)

El sistema se compone de tres contenedores principales:

- **Aplicación Web (Frontend):** Interfaz de usuario desarrollada en HTML/CSS/JavaScript, consumida desde el navegador. Se comunica con el backend mediante peticiones HTTP/REST.
- **Servidor de Aplicaciones (Backend):** Desarrollado en Spring Boot. Contiene los controladores REST, la lógica de triaje Manchester y la gestión de roles y sesiones.
- **Base de Datos (PostgreSQL):** Almacena de forma persistente los datos clínicos, usuarios, turnos y registros de triaje. Se comunica exclusivamente con el backend.

2.1 Diagrama de Contenedores (C4 – Nivel 2)

El sistema se compone de tres contenedores principales:

- **Aplicación Web (Frontend):** Interfaz de usuario desarrollada en HTML/CSS/JavaScript, consumida desde el navegador. Se comunica con el backend mediante peticiones HTTP/REST.
- **Servidor de Aplicaciones (Backend):** Desarrollado en Spring Boot. Contiene los controladores REST, la lógica de triaje Manchester y la gestión de roles y sesiones.
- **Base de Datos (PostgreSQL):** Almacena de forma persistente los datos clínicos, usuarios, turnos y registros de triaje. Se comunica exclusivamente con el backend.

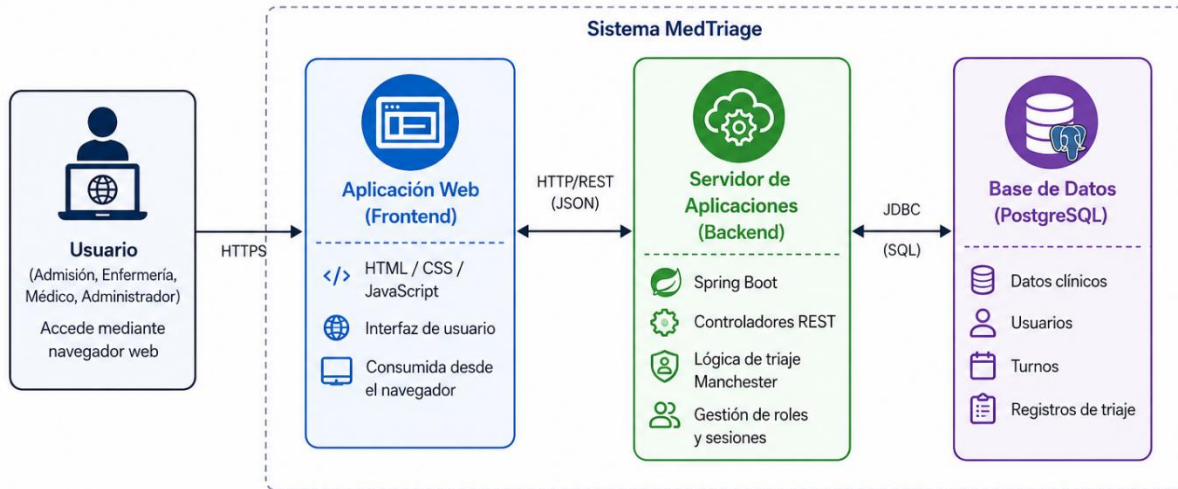


Figura 2 Diagrama de Contenedores

3. DESCRIPCIÓN DE COMPONENTES

3.1 Capa de Vista (Frontend)

Corresponde a la interfaz web con la que interactúan los usuarios del sistema. Está compuesta por los siguientes módulos:

- **Módulo de Autenticación:** Formulario de inicio de sesión y redirección según rol.
- **Módulo de Admisión:** Formulario de registro demográfico del paciente.
- **Módulo de Enfermería:** Formularios de signos vitales, discriminadores Manchester y confirmación de triaje.
- **Módulo Médico:** Vista de cola de pacientes, registro de evolución clínica y finalización de turno.
- **Módulo Administrador:** Generación y visualización de reportes estadísticos.

- **Pantalla Pública:** Vista de solo lectura con turnos llamados de forma anónima.

3.2 Capa de Controlador (Backend)

Corresponde al servidor de aplicaciones desarrollado en Spring Boot. Gestiona las peticiones del frontend, aplica las reglas de negocio y coordina el acceso a la base de datos. Sus componentes principales son:

- **Controlador de Autenticación:** Valida credenciales y gestiona sesiones por rol.
- **Controlador de Pacientes:** Gestiona el registro y consulta de datos demográficos.
- **Controlador de Triage:** Recibe signos vitales y discriminadores, ejecuta el algoritmo Manchester y retorna el nivel calculado.
- **Controlador de Cola:** Gestiona la inserción, ordenamiento y actualización de turnos en tiempo real.
- **Controlador de Reportes:** Genera las estadísticas operativas del servicio de urgencias.

3.3 Capa de Modelo (Base de Datos)

Corresponde a la base de datos relacional PostgreSQL. Almacena de forma persistente toda la información del sistema. Las entidades principales son:

- **Usuario:** Datos de acceso y rol asignado.
- **Paciente:** Datos demográficos y estado en el flujo de atención.
- **Signos Vitales:** Métricas clínicas registradas por enfermería.
- **Triage:** Nivel asignado, discriminadores seleccionados, justificación y trazabilidad.
- **Turno:** Código, nivel de prioridad, hora de llegada y estado de atención.
- **Reporte:** Registros estadísticos generados por el administrador.

4. DECISIONES DE ARQUITECTURA (ADR)

4.1 ADR-001 – Adopción del Patrón MVC

Se decidió utilizar el patrón Modelo-Vista-Controlador como estilo arquitectónico principal del sistema. Esta decisión responde a la necesidad de separar claramente la lógica de negocio, la capa de presentación y el acceso a datos, lo que facilita el mantenimiento, la escalabilidad y la organización del código a lo largo del desarrollo.

Como consecuencia de esta decisión, el frontend y el backend se desarrollan de forma independiente y se comunican mediante una API REST, descartando una arquitectura monolítica sin separación de capas que habría dificultado la evolución futura del sistema.

4.2 ADR-002 – Selección de Spring Boot como Framework Backend

Se decidió implementar el backend con el framework Spring Boot debido a que ofrece soporte nativo para la construcción de APIs REST, gestión de seguridad mediante Spring Security e integración directa con PostgreSQL, todo bajo una estructura MVC consolidada que reduce significativamente el tiempo de configuración inicial del proyecto.

Frente a alternativas como Node.js con Express o Django con Python, Spring Boot resultó más adecuado por su robustez en entornos empresariales y su compatibilidad con los requerimientos de seguridad y disponibilidad definidos en el SRS. Como consecuencia, el equipo de desarrollo requiere conocimientos en Java y el ecosistema Spring.

4.3 ADR-003 – Selección de PostgreSQL como Motor de Base de Datos

Se decidió utilizar PostgreSQL como sistema gestor de base de datos relacional, dado que los datos clínicos del sistema son de naturaleza estrictamente relacional: pacientes, turnos, triajes y usuarios mantienen relaciones directas entre sí que requieren integridad referencial y consistencia transaccional.

PostgreSQL ofrece además soporte para cifrado de datos en reposo, alto rendimiento en consultas concurrentes y cumplimiento con los estándares de seguridad exigidos por la Ley 1581 de 2012, lo que lo posicionó por encima de alternativas como MySQL o MongoDB para este contexto clínico.

4.4 ADR-004 – Comunicación mediante API REST con HTTPS

Se decidió que toda la comunicación entre el frontend y el backend se realizara mediante una API REST sobre el protocolo seguro HTTPS. Esta decisión garantiza el desacoplamiento entre capas, permite que ambos componentes evolucionen de forma independiente y asegura la confidencialidad de los datos clínicos transmitidos, cumpliendo con el RNF-002 definido en el SRS.

Como consecuencia, el despliegue en producción requiere un certificado SSL/TLS válido instalado en el servidor, descartando comunicación directa entre el frontend y la base de datos o el uso exclusivo de WebSockets, que no ofrecen el mismo nivel de seguridad y estandarización.

5. ATRIBUTOS DE CALIDAD DE LA ARQUITECTURA

5.1 Seguridad

La arquitectura garantiza la seguridad del sistema mediante el cifrado de datos sensibles en reposo con AES-256 y la transmisión segura de información a través de HTTPS. El control de acceso basado en roles impide que un usuario acceda a módulos fuera de su perfil, y toda modificación del triaje queda registrada con trazabilidad completa de usuario, fecha y hora.

5.2 Disponibilidad

El diseño del sistema contempla una disponibilidad mínima del 99,9% mediante la implementación de backups automáticos diarios y una infraestructura de hosting con capacidad de recuperación ante fallos. Dado el carácter crítico del servicio de urgencias, el sistema no puede permitirse períodos de inactividad prolongados.

5.3 Rendimiento

La arquitectura MVC con API REST permite distribuir la carga de procesamiento entre el cliente y el servidor, optimizando los tiempos de respuesta. El algoritmo de triaje debe responder en un máximo de 1,5 segundos y las interfaces críticas de cola de espera en menos de 2 segundos, conforme a lo establecido en los RNF-005 y RNF-006 del SRS.

5.4 Mantenibilidad

El desacoplamiento entre frontend y backend mediante API REST facilita la modificación independiente de cada capa sin afectar el funcionamiento del sistema completo. La estructura modular del backend en Spring Boot permite incorporar nuevos controladores o ajustar la lógica de triaje sin intervenir en la capa de presentación ni en la base de datos.